

Implement and train a Multilayer Feed-Forward Network (MLP) on the Wine Quality dataset

Aim:

- To implement a MLP on the **Wine Quality dataset**
- To experiment with:
 - Different **hidden layers**
 - Different **number of neurons**
- To analyze how architecture affects performance

Theory – no need to write

◆ 1. Multilayer Feed-Forward Network (MLP)

- Consists of:
 - Input layer
 - Hidden layers
 - Output layer
 - Learns complex patterns using **non-linear transformations**
-

◆ 2. Effect of Architecture

Hidden Layers:

- More layers → deeper representation
- Too many → overfitting

Neurons:

- More neurons → higher learning capacity
 - Too few → underfitting
-

◆ 3. Trade-off

Case	Result
Small network	Underfitting

Case	Result
Optimal network	Good performance
Large network	Overfitting

Algorithms

Step 1: Import Libraries

Step 2: Load Dataset

Step 3: Perform Preprocessing

Step 4: Function to Create Model

Step 5: Experiment with Architectures

Step 6: Compare Results

Program

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Input
from tensorflow.keras.utils import to_categorical
data = pd.read_csv("WineQT.csv")

X = data.drop('quality', axis=1).values
y = data['quality'].values
encoder = LabelEncoder()
y = encoder.fit_transform(y)
y = to_categorical(y)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=0)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
```

```

X_test = scaler.transform(X_test)
def create_model(hidden_layers):
    model = Sequential()
    model.add(Input(shape=(X_train.shape[1],)))

    for neurons in hidden_layers:
        model.add(Dense(neurons, activation='relu'))

    model.add(Dense(y.shape[1], activation='softmax'))

    model.compile(optimizer='adam',
                  loss='categorical_crossentropy',
                  metrics=['accuracy'])

    return model
architectures = {
    "Model 1 (Simple)": [8],
    "Model 2 (Medium)": [32, 16],
    "Model 3 (Deep)": [64, 32, 16]
}

results = {}

for name, layers in architectures.items():
    print("\nTraining", name)

    model = create_model(layers)

    history = model.fit(X_train, y_train,
                        epochs=50,
                        batch_size=32,
                        validation_split=0.2,
                        verbose=0)

    loss, acc = model.evaluate(X_test, y_test, verbose=0)
    results[name] = acc

    print(f"{name} Accuracy: {acc:.4f}")
names = list(results.keys())
values = list(results.values())

```

```
plt.bar(names, values)
plt.title("Model Accuracy Comparison")
plt.ylabel("Accuracy")
plt.xticks(rotation=20)
plt.show()
```